

DAY 25:
JAVASCRIPT
GARBAGE
COLLECTION &
MEMORY
MANAGEMENT



Lakshmiprasanna Vakada

JavaScript is a high-level language, meaning developers don't have to manually allocate and free memory. But how does JavaScript manage memory automatically? 🤔

The answer: Garbage Collection (GC)

◆ What is Garbage Collection?

Garbage Collection is the process of automatically reclaiming memory occupied by objects that are no longer in use.

- ✅ JavaScript engine (V8, SpiderMonkey, etc.) does this for us!
- ✅ It finds objects that are not reachable and frees up memory.

◆ How Memory Works in JavaScript?

Memory management in JavaScript consists of 3 key steps:

- 1** Memory Allocation → Memory is allocated when we create variables, objects, functions, etc.
- 2** Memory Usage → JavaScript runs operations using allocated memory.
- 3** Memory Cleanup (Garbage Collection) → Unused memory is automatically freed.

◆ Example of Memory Allocation & Cleanup

```
function createUser() {  
  let user = {  
    name: "Alice",  
    age: 25,  
  };  
  console.log(user.name); // "Alice"  
}  
  
createUser(); // user object is created  
in memory  
// After the function finishes, 'user'  
is no longer accessible → garbage  
collected
```

Here, the user object is created inside the function. Once the function finishes execution, user is no longer needed and gets garbage collected. 

◆ Reachability & Garbage Collection

Garbage Collector removes objects that are **NOT REACHABLE**.

An object is considered reachable if:

- ✓ A global variable references it
- ✓ A function is actively using it
- ✓ It is part of an active closure

Example: Unreachable Object

```
let user = {  
  name: "Alice",  
};  
user = null; // The object { name:  
"Alice" } is now unreachable and will be  
garbage collected.
```

- ◆ Since user is set to null, the original object is no longer referenced and will be garbage collected.
- ◆ JavaScript Garbage Collection Algorithm:
Mark-and-Sweep 🖌️

Most modern JavaScript engines use the Mark-and-Sweep algorithm:

- 1 Mark Phase → The engine marks all reachable objects.
- 2 Sweep Phase → Unmarked (unreachable) objects are removed from memory.

This happens automatically in the background.

◆ Common Memory Leaks in JavaScript (⚠️ Avoid These!)

1 Unused Global Variables

● Bad:

```
let unusedVar = "I will never be used";
```

✓ Fix:

Remove unused variables or use let/const inside functions to limit their scope.

2 Accidental Global Variables (Without let or const)

 Bad:

```
function createVar() {  
    someVar = "Oops, global!";  
}  
createVar();
```

 Fix:

Always declare variables using let or const.

3 Forgotten Event Listeners (Leaking Memory)

● Bad:

```
document.getElementById("btn").addEventListener("click", function () {  
    console.log("Clicked!");  
});
```

✓ Fix: Remove event listeners when no longer needed:

```
const btn = document.getElementById("btn");  
function handleClick() {  
    console.log("Clicked!");  
}  
btn.addEventListener("click", handleClick);  
btn.removeEventListener("click", handleClick);
```

4 Closures Holding References

● Bad:

```
function outer() {  
  let bigObject = new  
Array(1000000).fill("data"); // Large  
memory object  
  return function inner() {  
    console.log(bigObject[0]); //  
`bigObject` is never garbage collected  
  };  
}  
const holdReference = outer();  
// Memory leak
```

✓ Fix:

Nullify references when no longer needed.

```
holdReference = null;
```

◆ Best Practices for Memory Management in JavaScript

- ✓ Use `let` and `const` to avoid accidental globals
- ✓ Nullify references when objects are no longer needed
(`myObj = null`)
- ✓ Remove event listeners when they are no longer required
- ✓ Avoid memory-heavy closures
- ✓ Use tools like Chrome DevTools (Memory Tab) to track memory usage